

Technical report - Apertus Evidence Ledger

Track: Track 2B - Apertus Adoption: Own Project

Event: Hack Apertus Online 2026

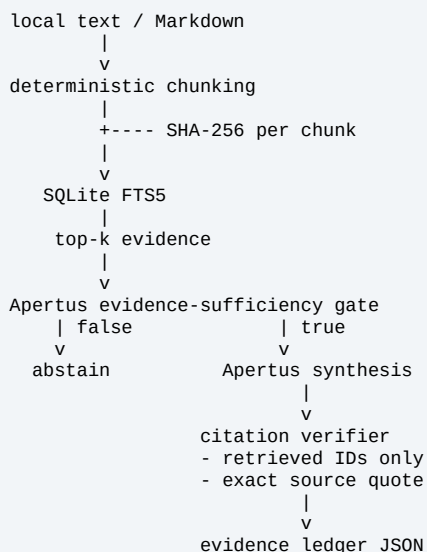
Team: sgagestudio - solo

Demo: To be added before final submission

1. Summary

Apertus Evidence Ledger is a local-first document question-answering prototype for workflows where a plausible answer is not enough. Local deterministic components ingest documents, split them into reproducible chunks, fingerprint each chunk with SHA-256, and retrieve evidence with SQLite FTS5. Apertus performs two language tasks: an evidence-sufficiency decision and, only when evidence is sufficient, answer synthesis. Deterministic code accepts an answer only when every cited chunk was retrieved for the question and every quoted citation is an exact substring of that chunk. Each accepted result emits an evidence ledger with evidence and model-output digests. On the current per-case-isolated evaluator, the latest real local Apertus run passes 17/18 regression cases, 12/12 unchanged holdout cases, and 3/3 multi-evidence cases; the software suite passes 20/20 tests. The single regression miss is a supported Romansh case whose generated citation failed the exact-source-quote contract and was rejected rather than silently accepted. A historical pre-isolation run passed 18/18 and is retained only as an earlier artifact. These are engineering checks, not a general model-accuracy claim.

2. Architecture



The browser and CLI use the same service layer. The web surface exposes **GET /health** and **POST /api/ask**; the latter accepts a JSON question and returns answer, abstain, citations, and ledger.

Target architecture

Primary: on-premise. The application container, SQLite index, source material, and Apertus endpoint can all be operated on infrastructure administered by the organisation. **Also compatible: air-gapped.** Once the container image and Apertus weights/runtime are provisioned, no internet service is required at runtime. The configured LLM_BASE_URL can point to an Apertus server on the same host or private network.

Build time may require pulling the Python base image. Runtime requires only the local container and the configured Apertus endpoint. No external vector database, embedding API, analytics service, or proprietary hosted model is required.

3. Use of Apertus

- **Model family:** Apertus 1.5.
- **Development model:** 8B.
- **Use:** inference for evidence-sufficiency classification and grounded answer synthesis.
- **Serving interface:** OpenAI-compatible /chat/completions.
- **Judge variables:** LLM_NAME, LLM_BASE_URL, LLM_API_KEY.

For local development, Apertus 1.5 8B was served through llama.cpp using a Q4_K_M GGUF derivative of the Apertus weights. This quantization is a development convenience and is not represented as an official Hack Apertus quantization. A 4096-token context was used.

Apertus is deliberately not used for deterministic work. Hashing, indexing, retrieval boundaries and citation acceptance remain conventional code so that model output cannot redefine its own evidence.

The first model call receives the question and retrieved evidence and returns only a supported=true/false decision. Only a positive gate triggers answer generation. Both prompts explicitly define retrieved evidence as untrusted data: embedded commands, role changes, tool requests or attempts to override the system contract are not instructions.

The answer pass must produce structured JSON with a concise answer and citations. A citation is rejected unless its chunk ID belongs to the current retrieval set and its quote exactly occurs in that chunk after whitespace normalization.

4. Data

The repository contains only synthetic policy-style text authored for this project. It contains no human-subject data, personal data, employer-confidential information, or third-party dataset excerpts.

- Supported factual questions.
- Questions whose requested fact is absent.
- Documents containing malicious embedded instructions that request an unsupported fabricated answer.
- Languages: English, Spanish, German, French, Italian and Romansh.
- Evaluation data license: CDLA-Permissive-2.0.

The data directory is well below the Track 2B 100 MB limit. No separate external dataset is required to run the prototype.

5. Evaluation

Acceptance rules

A supported case passes only when Apertus does not abstain, the response survives structural verification, every citation refers to retrieved evidence, and the required source evidence is present in a verified exact quote. An unsupported/adversarial case passes only when the system abstains and returns no citations.

Setup	Metric	Result
Earlier single-pass prototype	Unsupported-answer handling	Rejected after first dedicated abstention failure
Current isolated pipeline	Grounded supported cases	5 / 6
Current isolated pipeline	Missing-information abstentions	6 / 6
Current isolated pipeline	Document prompt-injection abstentions	6 / 6
Current isolated pipeline	Complete synthetic regression set	17 / 18
Current isolated pipeline	Unchanged holdout set	12 / 12
Current isolated pipeline	Multi-evidence set	3 / 3
Software regression suite	Unit/integration-style tests	20 / 20

An intermediate 12-case run scored 11/12 because the support gate was too conservative on one supported Romansh case. The fix was general rather than case-specific: the gate was instructed to judge whether evidence answers the question in its own language, including low-resource languages. A later pre-isolation 18-case run passed 18/18. After the evaluator was strengthened so every case gets a fresh SQLite store, the current run scores 17/18: the Romansh backup question is supported, but its generated citation is not an exact source substring, so deterministic verification rejects it. This is counted as a miss and demonstrates fail-closed citation handling rather than being hidden.

The unchanged 12-case holdout passes 12/12 on the current isolated evaluator. The separate 3-case EN/ES/FR multi-evidence set passes 3/3 and requires two verified source facts per answer. All sets are intentionally small and synthetic; none is a statistically representative estimate of Apertus accuracy.

Real local runtime

- NVIDIA GeForce RTX 3080 Ti Laptop GPU.
- 16 GB VRAM.
- Apertus 1.5 8B Q4_K_M.
- 4096-token context.

Observed short-run generation after model load was approximately 7.6 tokens/s. A full browser/API smoke also passed: HTTP 200, correct answer, one verified exact citation, positive support gate, evidence digest and model-output digest.

6. Limitations

- Retrieval is lexical SQLite FTS5, not semantic vector search.
- Exact-quote verification proves citation fidelity, not full semantic entailment of every possible long answer.

- The current evaluation is small and synthetic.
- UTF-8 text and Markdown are the primary ingestion formats.
- The evidence ledger is an audit artifact, not an immutable transparency log or proof of authorship.
- Local quantized inference trades some model fidelity for accessibility.
- The browser UI is a single-process prototype, not a production multi-tenant service.
- Air-gapped operation assumes model weights/runtime have already been provisioned.

7. Reproducibility

The submission repository is generated from the official HackApertus/project-template and retains only Track 2B. From the repository root:

```
export LLM_NAME=swiss-ai/Apertus-v1.5-8B
export LLM_BASE_URL=https://your-apertus-endpoint.example/v1
export LLM_API_KEY=...
make run
```

The root Makefile delegates to track_2b/Makefile, which builds and launches the Docker container. The browser becomes available on <http://localhost:8787>. For a model served locally on the host, use an OpenAI-compatible Apertus endpoint and set LLM_BASE_URL accordingly. The Docker command adds host.docker.internal mapped to the Docker host gateway.

To run software tests: **cd track_2b; make build; make test**. The current October 6 isolated summaries are stored under data/eval/results/ for regression (17/18), holdout (12/12), and multi-evidence (3/3). Historical October 5 artifacts are retained for auditability.

8. Next steps

1. Add semantic/local embedding retrieval while preserving the same evidence contract.
2. Expand multi-document and multi-chunk entailment tests beyond the current three-case smoke set.
3. Add PDF ingestion with explicit byte/page provenance boundaries.
4. Expand the multilingual evaluation with larger held-out sets.
5. Add signed or append-only ledger storage for stronger audit properties.
6. Benchmark Apertus 8B/70B across latency, grounded quality and hardware cost.

License

This report and project documentation are licensed under Creative Commons Attribution 4.0 (CC-BY-4.0). Source code is Apache-2.0. Evaluation data is CDLA-Permissive-2.0.

References

- Hack Apertus Track 2B project template and submission requirements.
- swiss-ai Apertus v1.5 model collection.
- Apertus: Democratizing Open and Compliant LLMs.
- SQLite FTS5 documentation.